Object Oriented Analysis and Modelling

Learning outcomes

- Define Object modelling and benefits
- Define and apply OOAD concepts
- Recognize and understand the basic concepts and constructs of object modelling
- Define the UML and its various types of diagrams
- Evolve business requirements use-case model into a systems analysis use-case model
- Construct an activity diagram
- Discover objects, classes and their relationships
- Construct a class diagram

Benefits of Object-oriented analysis

- Better code reuse
- Geographically separated groups of programmers can collaborate easily
- Modularity for easier troubleshooting
- Flexibility through polymorphism
- Effective problem solving

Object Modelling

- **Object modeling** – a technique for identifying **objects** within the systems environment and the **relationships** between those objects.
- Object oriented analysis is concerned with defining **static** structure and **dynamic** behavior models of information systems instead of data and process models.

UML

- **Unified Modeling Language** (UML) – a set of modeling conventions that is used to specify or describe a software system in terms of objects.
- The UML does not prescribe a method for developing systems—only a **notation** widely accepted as a standard for object modeling.

Object Oriented Analysis

- Objects and Classes
  - Attributes
  - Methods

Objects and Attributes

- **Objects** (Instances of Classes)
  - Something that is or is capable of being seen, touched, or otherwise sensed and about which users store data and associate behavior
  - **Types of objects**
    - Person – e.g. employee, customer, instructor, student
    - Place – e.g. warehouse, building, room, office
    - Thing – e.g. product, vehicle, computer, videotape
    - Event – e.g. an order, payment, invoice, application
    - Sensual – e.g. phone call, meeting

Attributes and Services

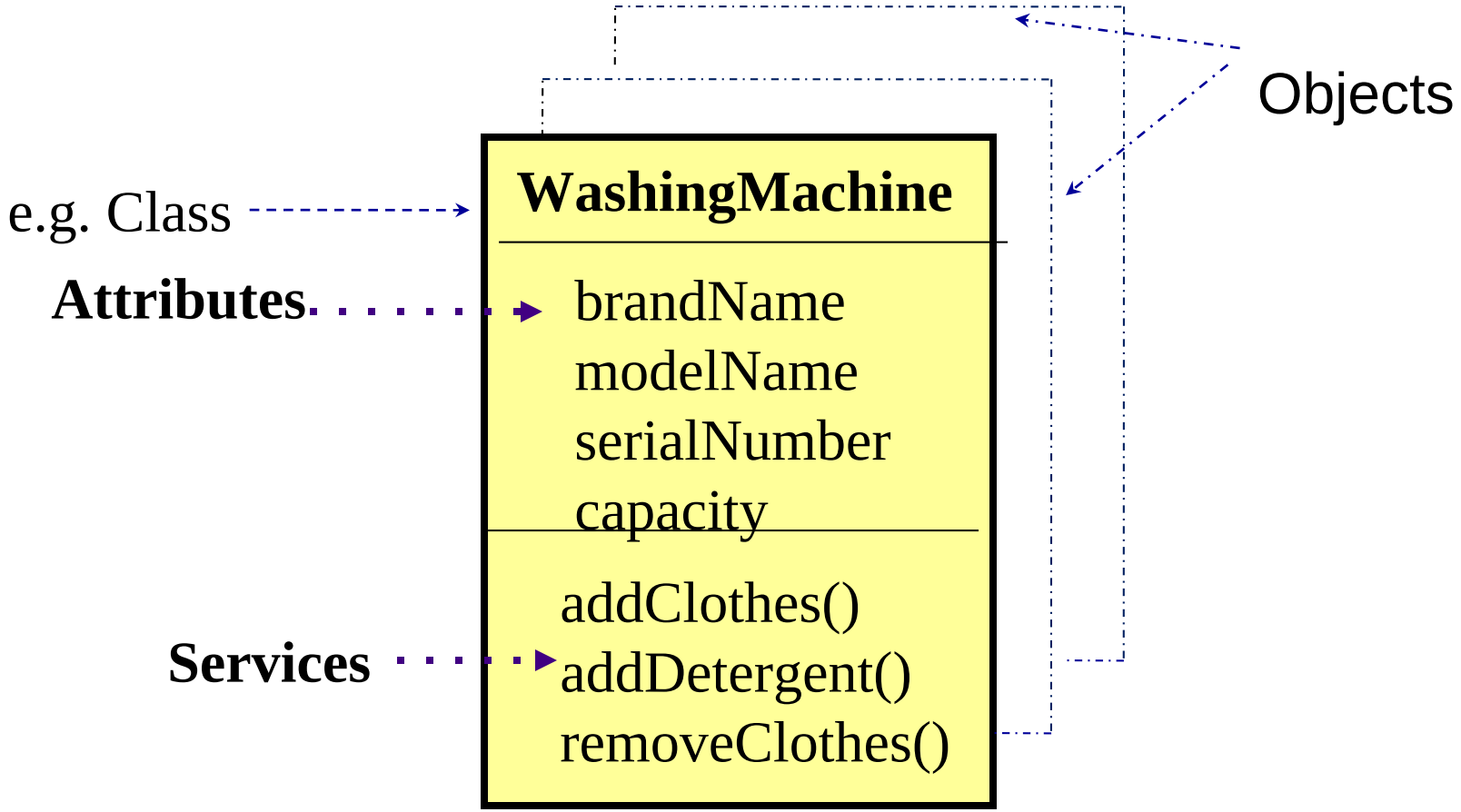
- **Attributes** : The data that represents characteristics of interest about an object.

- **Services** : The set of things that the object can do correspond to functions that act on the object's data or attributes.

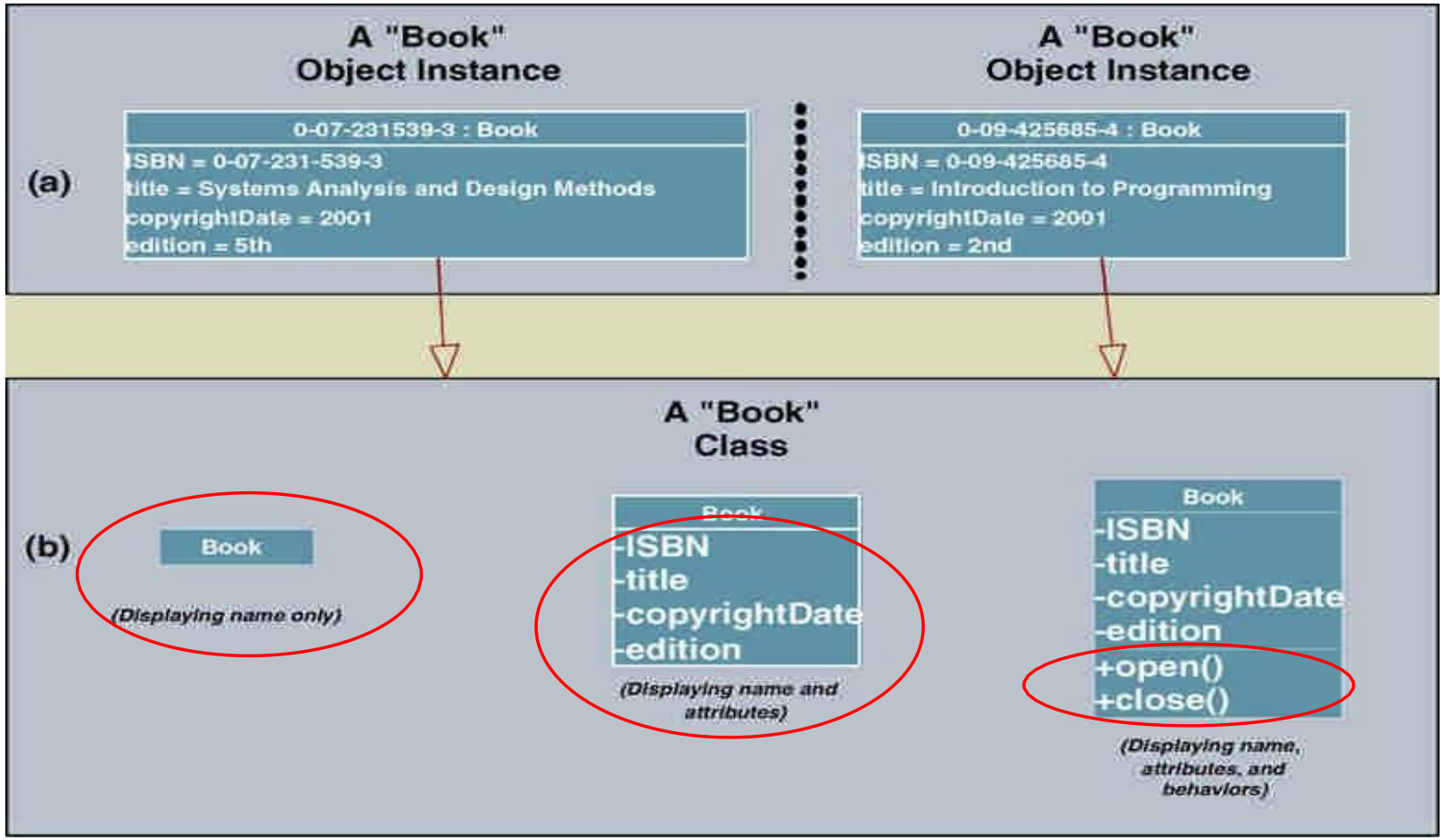
Class

- Class
  - A class is a generalized description that describes the collection of similar objects.
  - *Classes* in a library system - *Book, Copy, Member etc. (problem Domain Classes)*

Class and Objects



Representing Classes in UML



Abstraction

- A mechanism to reduce and filter out details so that one can focus on a few concepts at a time. Focus on Essentials
- Filter out an object's properties and operations until just the ones you need are left.

  Ignore the irrelevant. Ignore the unimportant.

  eg. Book – ~~Bookshop you bought~~

Object oriented concepts

- Inheritance
- Encapsulation
- Polymorphism

Inheritance

- The **concept** wherein methods and/or attributes defined in a class can be inherited or reused by another class.

  e.g. PERSON and STUDENT class

  PERSON and TEACHER class

  some individuals in the room might be classified as STUDENTS and TEACHERS. Thus, STUDENT and TEACHER classes are members of the class PERSON

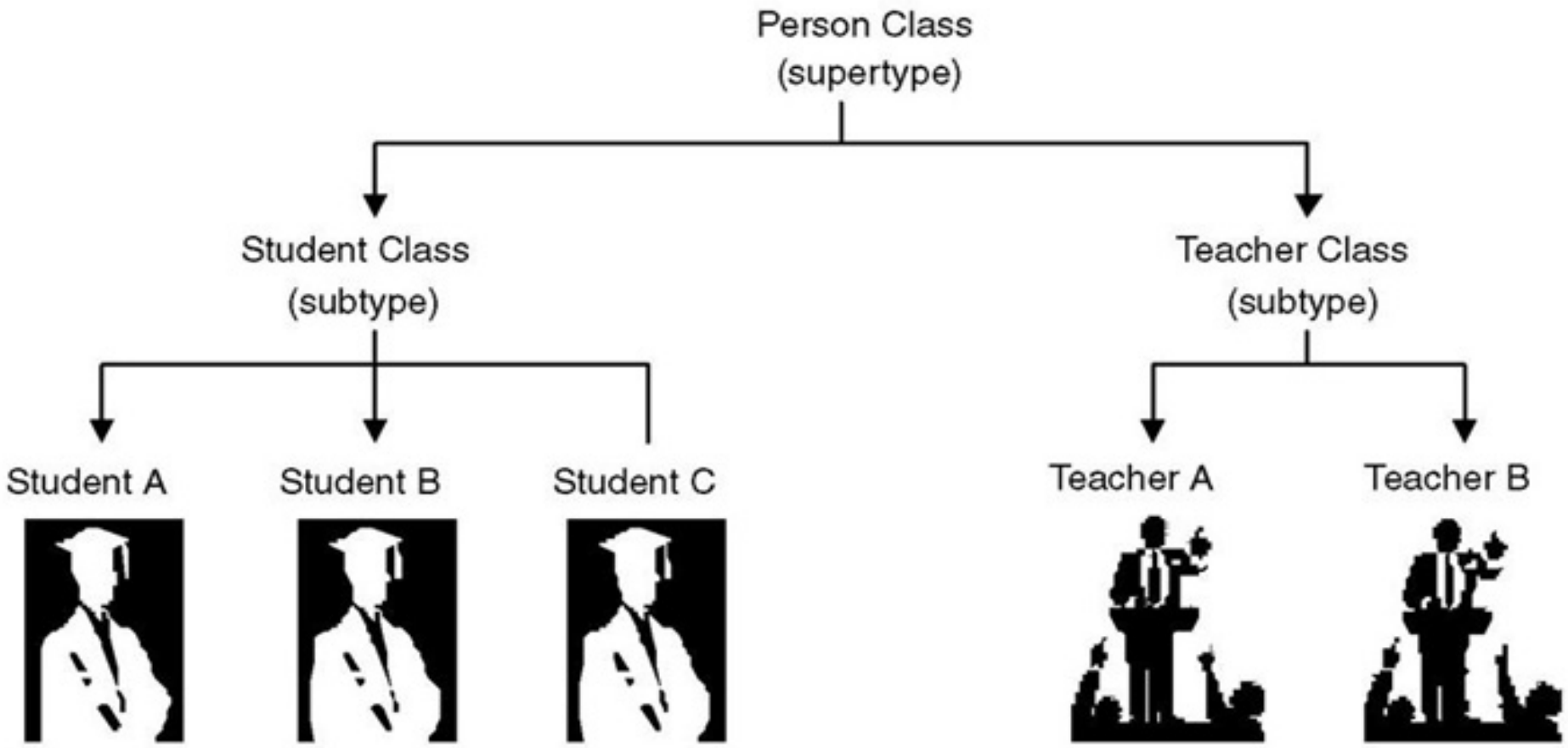
Generalization / Specialization

- A **technique** wherein the attributes and behaviors that are common to several types of classes are grouped / abstracted into their own class called a supertype.

- **Supertype** – an entity that contains attributes and behaviors common to one or more class subtypes. Also referred to as ***abstract*** or ***parent*** class.

- **Subtype** – Class that inherits attributes and behaviors from a supertype class.
  - Can contain other attributes and behaviors *unique* to it.
  - Also referred to as a ***child*** .

Inheritance cont….



Generalization / Specialization

Generalization

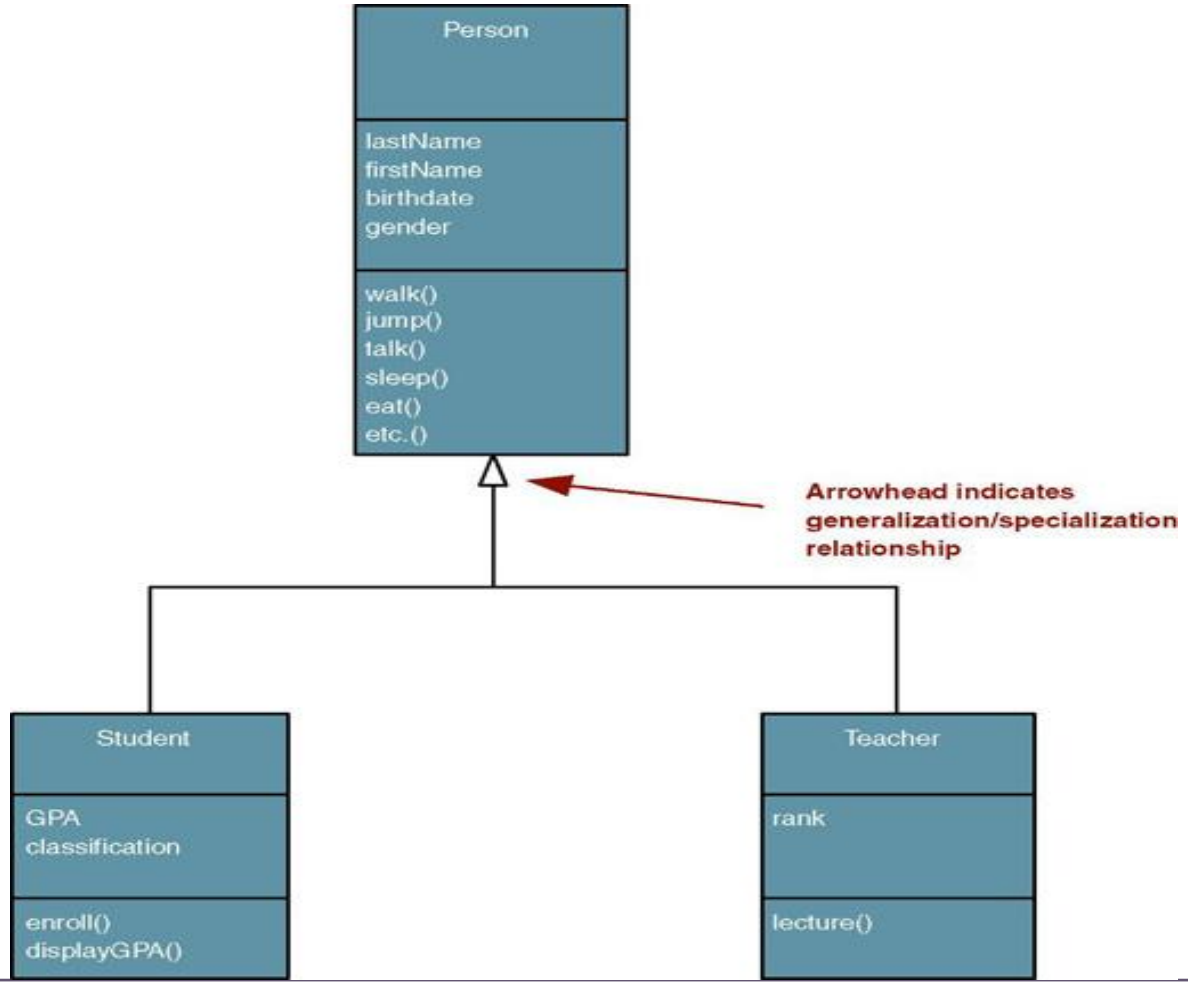
Person
firstName lastName birthdate gender
Walk() Jump() Talk() Sleep()

Specialization

Student
GPA Classification
Enroll() displayGPA()

Teacher
rank
Lecture()

UML Representation of Generalization/Specialization



Generalization / Specialization

Generalization

Person
firstName lastName birthdate gender
walk jump talk sleep

Inheritable
<u>Attributes</u>
And
<u>behavior</u>

Specialization

Student
GPA Classification
enroll displayGPA

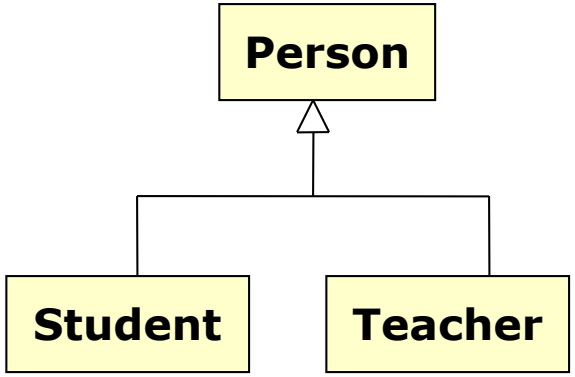
Teacher
rank
lecture

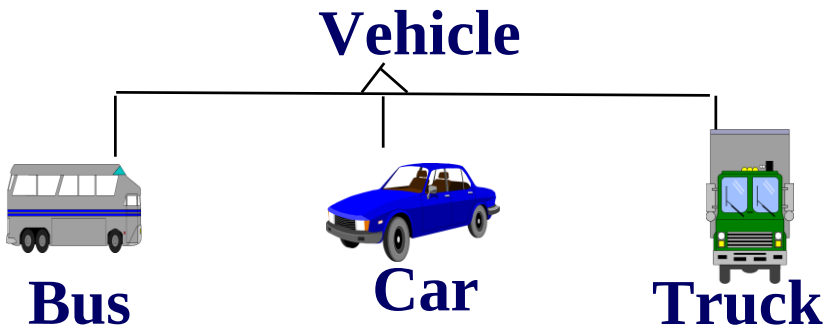
+

firstName
lastName
birthdate
gender
walk
jump
talk
sleep

Generalization / Specialization

- Generalization / Specialization





*** Specialized classes inherited from the parent class**

Polymorphism

- Concept that different objects can respond to the same message in different ways.
  - It allows many forms of the same service to be defined.
  - Sometimes an operation has the same name in different classes.

    eg. Open()

    open a door, open a window, open a bank account.

In each case, you are performing a different operation.

Polymorphism

- In object-orientation, each class “knows” how that operation is supposed to take place.
- *Related terms used in OO programming language*

  Function Overloading,

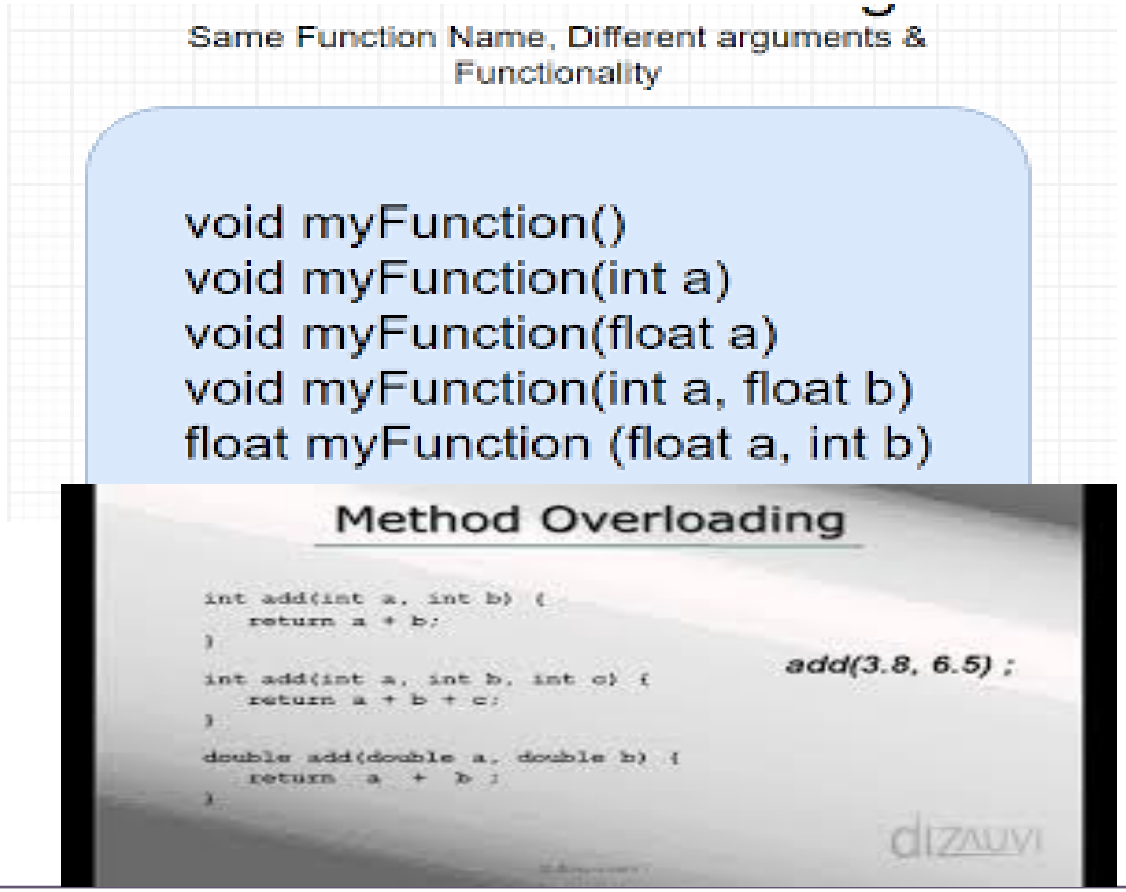
  Operator Overloading,

  Method Overriding

Polymorphism

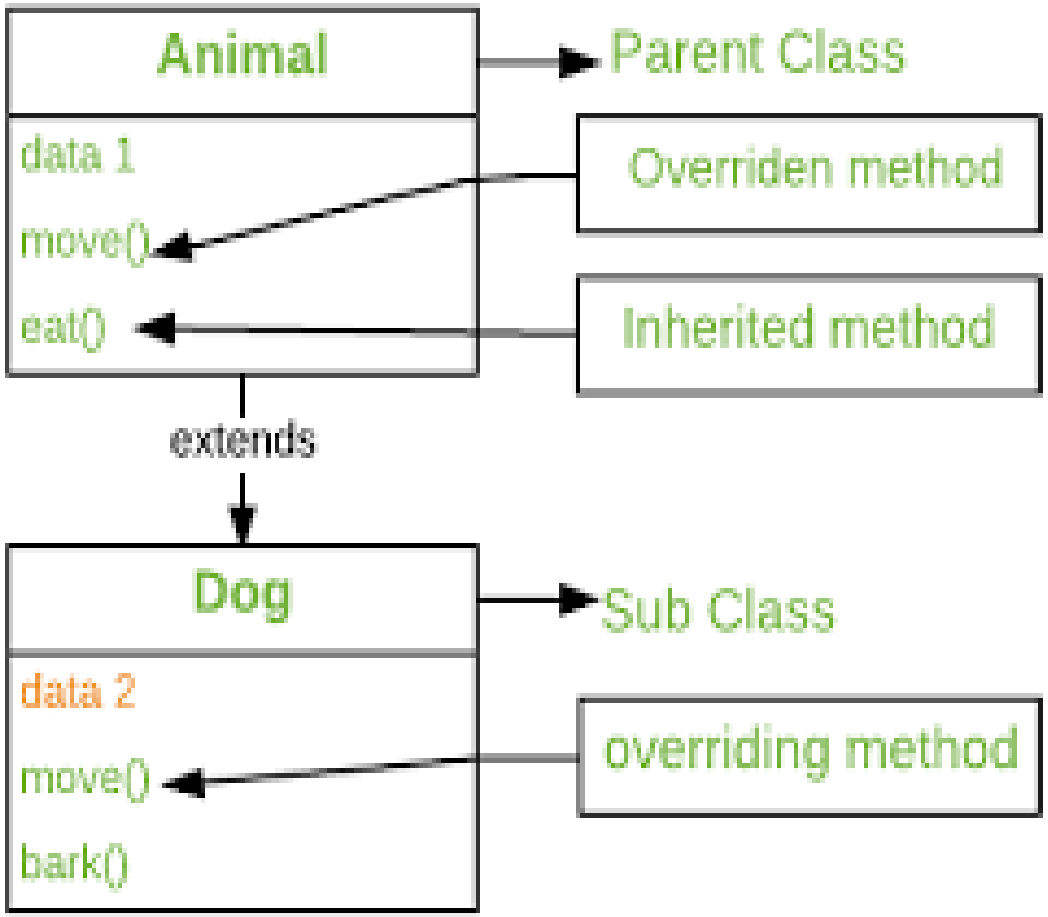
- Function(Method) Overloading

In some programming languages, **function overloading** or **method overloading** is the ability to create
multiple functions of the same name
with different implementations.

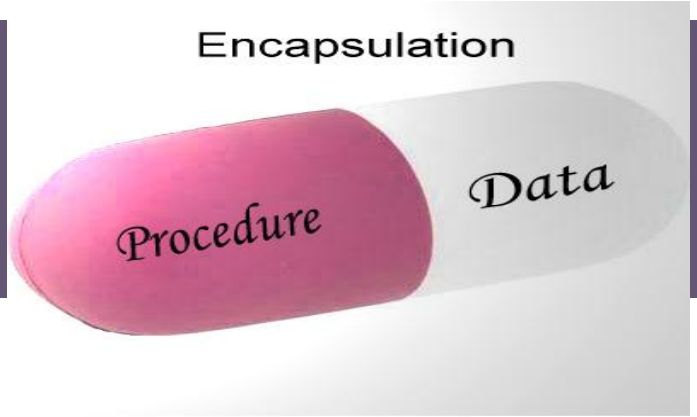


Polymorphism

Overriding – a technique whereby a subclass (subtype) uses an attribute or behavior of its own instead of an attribute or behavior inherited from the class (supertype).



Encapsulation



- **Packaging of several items together into one unit (both attributes and behavior of the object), Also protects the contents.**
- **The only way to access or change an object's attribute is through that object's specific behavior.**
- **Objects *encapsulates* what they do.**
  - That is, they hide the inner workings of their operations
    - **from the outside world**
    - **and from other objects**

Eg. Class BankAcount
{
Private: Balance
……
Withdraw(int)
Deposit(int)
}

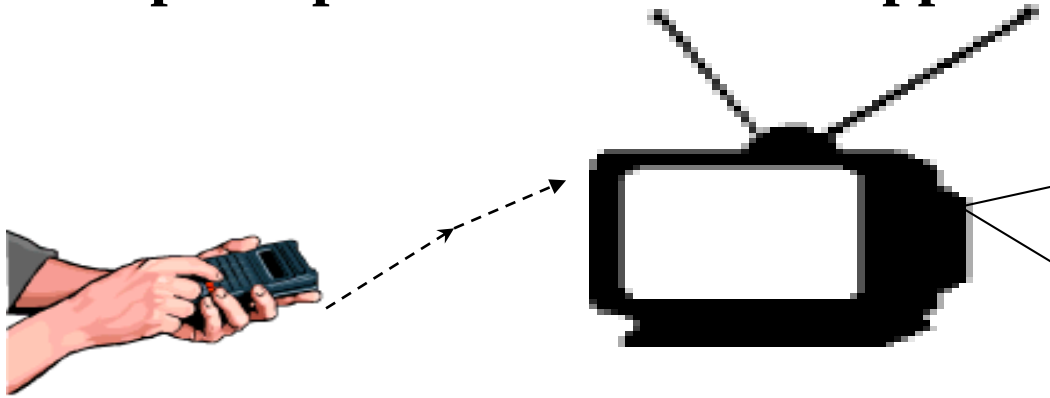
Encapsulation

Encapsulation- When an object carries out its operations, those operations are hidden.

E.g. When most people watch a television show,

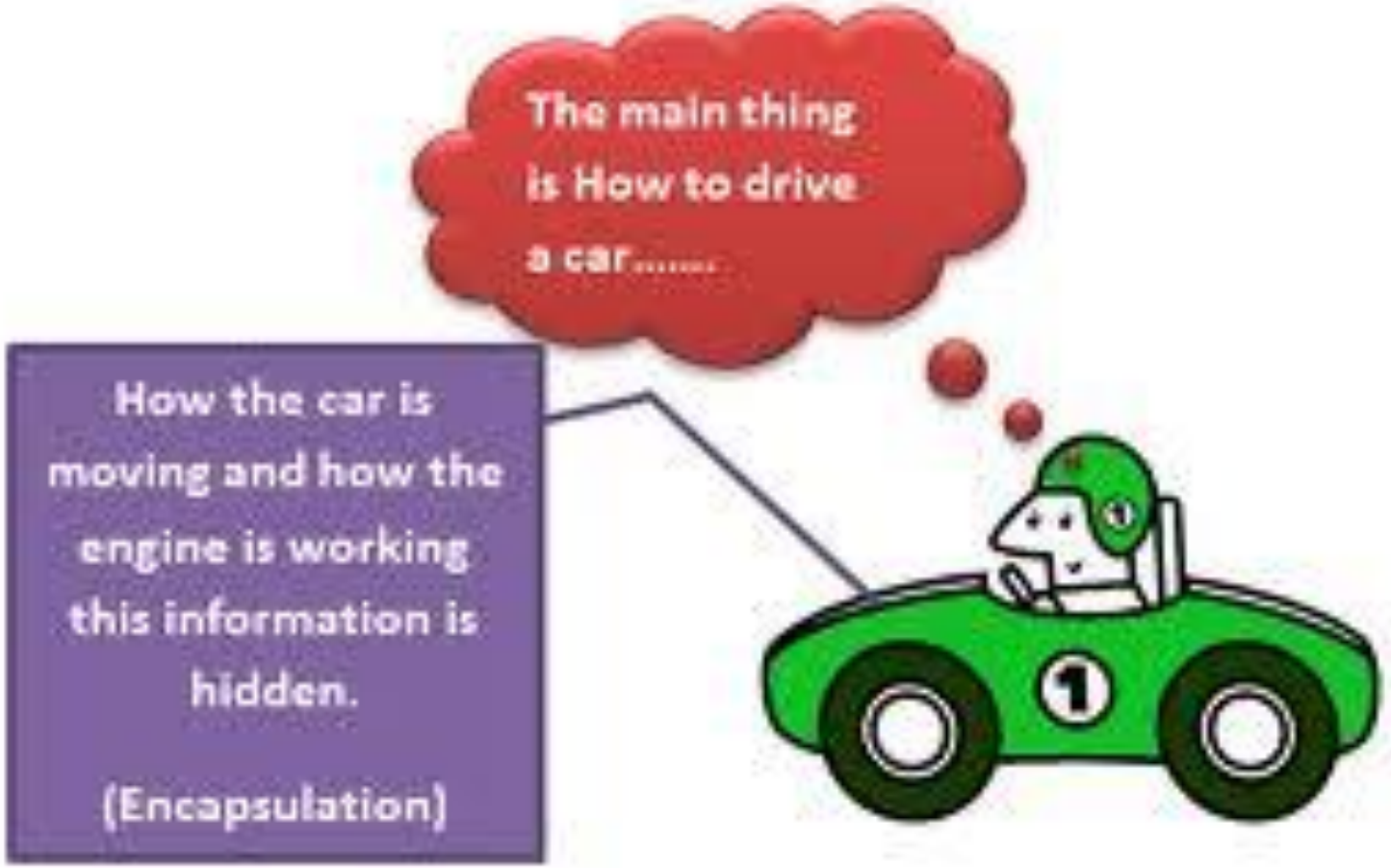
- they usually don't know or care about the complex electronics that sit in back of the TV screen

- or the complex operations that are happening.



The TV hides its operations from the person watching it.

Encapsulation



Encapsulation

- *Why important?*
  - In the software world, encapsulation helps cut down on the potential for bad things to happen.
  - In a system that consists of objects, the object depends on each other in various ways.
  - If one of them happen to malfunction, software engineers have to change it in some way.
  - Hiding its operations from other objects means it probably won’t be necessary to change those other objects.

Encapsulation

- An object hides what it does from other objects and from outside world
- But an object has to present a “face“ to the outside world, so you can initiate those operations.

  eg. A TV, has a set of buttons either on

  the TV or on a remote.

  The TV’s buttons are called *interfaces*.

Encapsulation

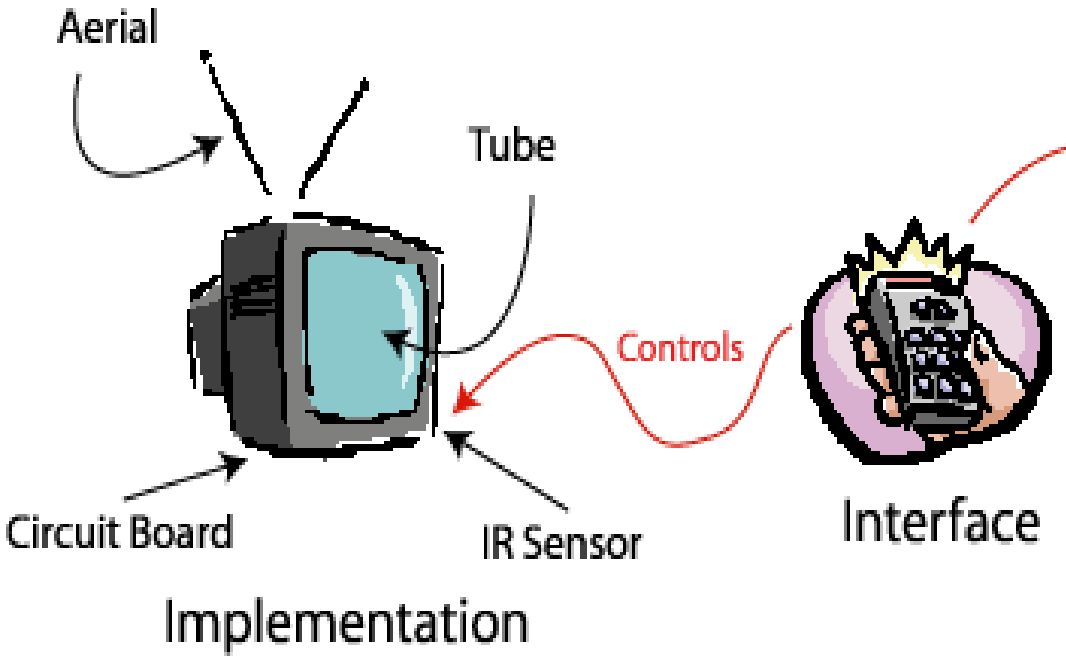
- Turning from software to reality, you see the importance of *encapsulation*.

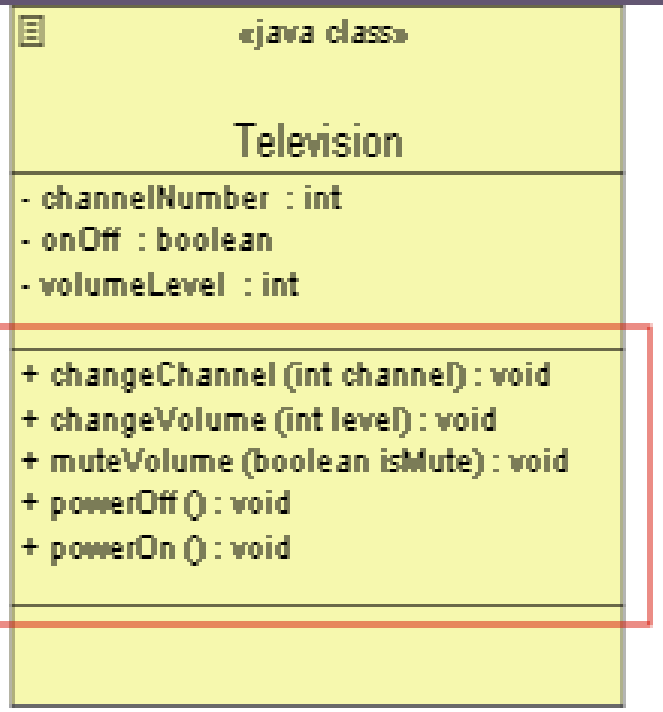
eg. Your computer monitor, hides its operations from your computers CPU.

When something goes wrong with your monitor, you either fix or replace it.

You probably won’t have to fix or replace the CPU along with it.

Encapsulation



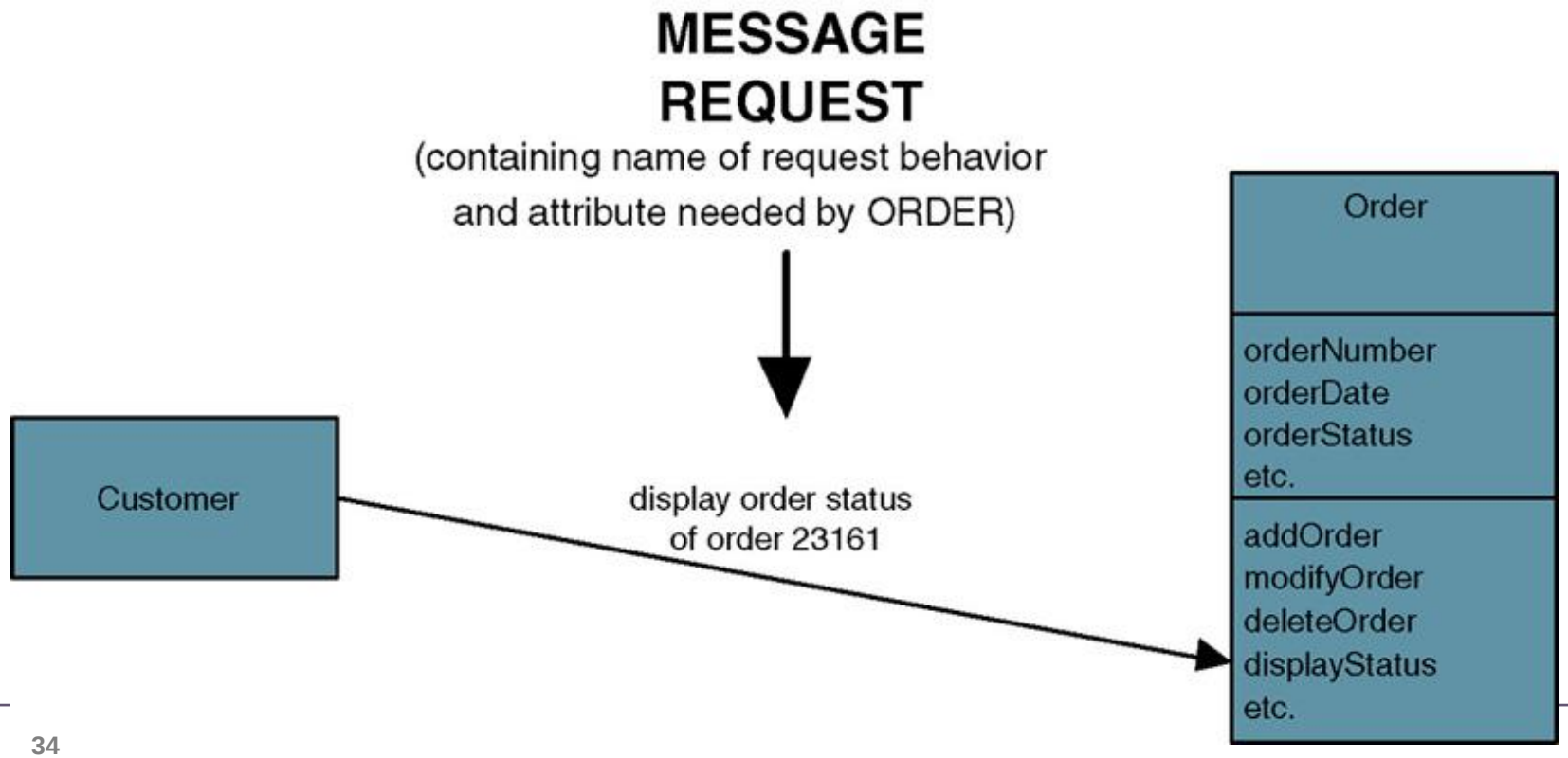


Encapsulation

- Encapsulation is done through the definition of *region of access*.
- *Region of access* defines the accessibility of the *services* or *attributes* in a class.
- In C++ (OO programming Language)
  - 3 types of access regions: *private, public, protected*
- *In Java*
  - *Access modifiers: default, public, private, protected,*
  - *Static modifier, final modifier, synchronized modifier, native modifier*

Message

Message – communication that occurs when one object invokes another object's method (behavior) to request information or some action



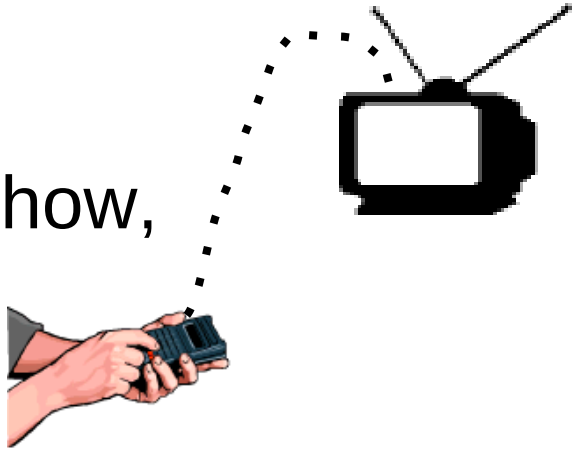
Message Passing

- Message Passing
  - In a system, objects work together.
  - They do this by sending messages to one another.
  - One object sends a message to another to perform an operation.
  - The receiving object performs that operation.

    eg. A TV and a Remote

Message Passing

- Message Passing
  - When you want to watch a TV show,
    - You hunt round for a remote,
    - Settle into your favorite chair and
    - Push the *On* Button.



  - What happens?
    - The remote object sends a message (literally!) to the TV object to turn itself on.
    - The TV object receives the message.
    - It knows how to perform the turn-on operation and turns itself on.

Message Passing

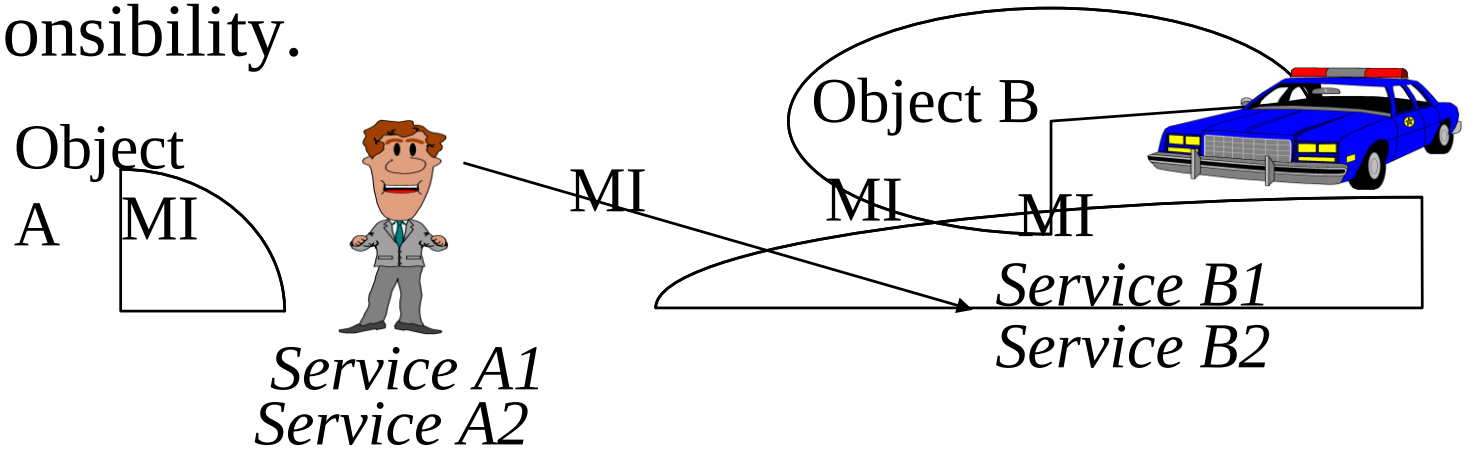
- Message Passing
  - When you want to watch a different channel,
    - You click the appropriate button on the remote,
    - Remote object sends a different message *change channel* to the TV object.
  - The remote can also communicate with the TV via other messages for *changing the volume, muting the volume etc.*.

Message Passing

- Message Passing
  - Most of the things you do from the remote, you can also do by *getting out of the chair*, *going to the TV*, and *clicking buttons on the TV.*
  - The interface the TV presents to you is obviously not the same interface it presents to the remote.

Message Passing

- Each object offers services within the realm of its responsibility.



* Objects use other objects services through *<u>message passing</u>*

* An object may even call its own services

Summary

- Many benefits of OOA and Modelling
- Several OOAD concepts.
    - Inheritance
    - Polymorphism
    - Encapsulation